

Prelab 8.3 Classifying Air-tight Vacuum and Air-leak Vacuum Data using Autoencoders for Anomaly Detection: Y and Z-axis

```
In [5]: # Let's check the installed tensorflow version
import tensorflow as tf

print('TensorFlow Version is', tf.__version__)
```

TensorFlow Version is 2.19.0

```
In [6]: # required Python packages for this colab
!pip install matplotlib
!pip install pandas
!pip install scipy
!pip install scikit-learn
```

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (1.16.3)
Requirement already satisfied: numpy<2.6,>=1.25.2 in /usr/local/lib/python3.12/dist-packages (from scipy) (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)

```
In [7]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```

import scipy.fftpack
from tensorflow import keras

from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_curve, auc
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers, losses
from tensorflow.keras.models import Model

```

```

In [8]: # Copying raw data from github dataset file
url = 'https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/main/lab/lab8/Prelab8_data.csv'
#df is the variable where the data is stored
df = pd.read_csv(url)

#Data selection
# X-axis: 'Xacc array [m/s2]'
# Y-axis: 'Yacc array [m/s2]'
# Z-axis: 'Zacc array [m/s2]'
# If you want to use x-axis (X-axis),
# AXIS = 'Xacc array [m/s2]'
AXIS = 'Yacc array [m/s2]' #Pick and write the axis you want to work <-----

#Exploding the values contained in selected column and converting the string values into float values
df_new = pd.concat([df['Condition'],df[AXIS].str.split(' ', expand=True).astype(float)], axis=1)
ds = df_new.copy()
#Converting the Classifier into binary values
ds.loc[df['Condition'] == 'Vacuuming', 'Status'] = 1
ds.loc[df['Condition'] == 'Air_leakage', 'Status'] = 0
ds.drop('Condition', axis=1, inplace=True)

#Data transformation

raw_data = ds.values
# The last element contains the labels
labels = raw_data[:, -1]

# The other data points are the vacuum accelerometer data
data = raw_data[:, 0:-1]

train_data, test_data, train_labels, test_labels = train_test_split(
    data, labels, test_size=0.2, random_state=21
)
#Normalizing the values of the dataset
min_val = tf.reduce_min(train_data)
max_val = tf.reduce_max(train_data)

train_data = (train_data - min_val) / (max_val - min_val)
test_data = (test_data - min_val) / (max_val - min_val)

train_data = tf.cast(train_data, tf.float32)
test_data = tf.cast(test_data, tf.float32)
#Splitting the dataset based on classification: train_labels: Vacuuming, ~train_labels: Air Leakage
train_labels = train_labels.astype(bool)
test_labels = test_labels.astype(bool)

normal_train_data = train_data[train_labels]
normal_test_data = test_data[test_labels]

anomalous_train_data = train_data[~train_labels]
anomalous_test_data = test_data[~test_labels]

portion_of_anomaly_in_training = 0.1 #10% of training data will be anomalies
end_size = int(len(normal_train_data)/(10-portion_of_anomaly_in_training*10))
combined_train_data = np.append(normal_train_data, anomalous_test_data[:end_size], axis=0)
combined_train_data.shape

```

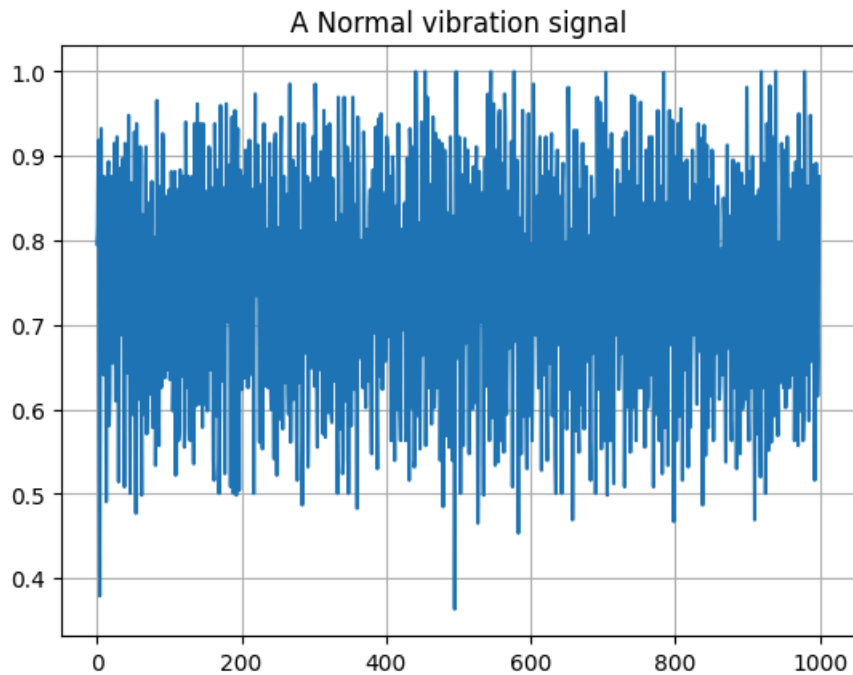
Out[8]: (536, 1000)

```

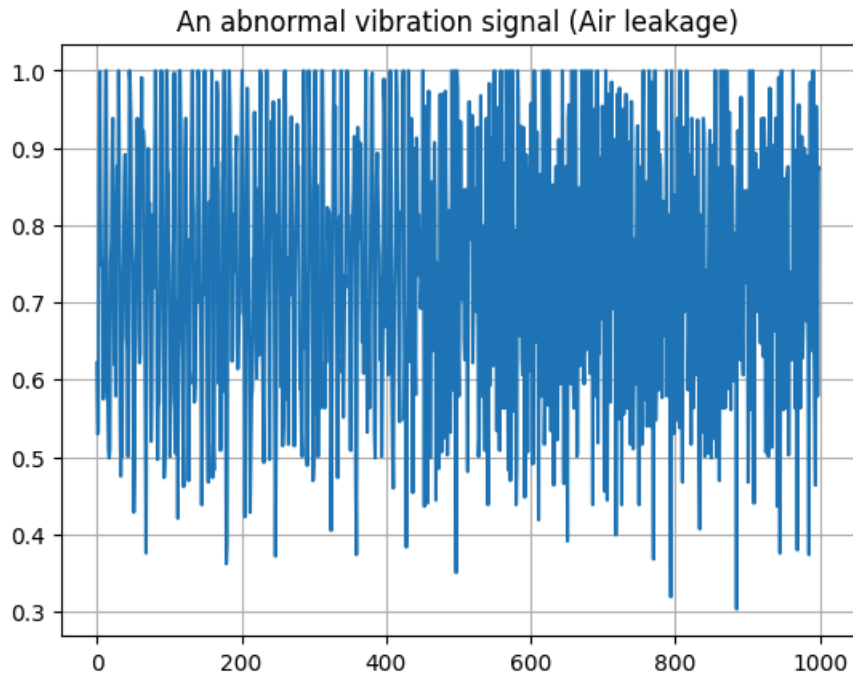
In [9]: #Plotting sample of normal data
plt.grid()
plt.plot(np.arange(1000), normal_train_data[0])

```

```
plt.title("A Normal vibration signal")
plt.show()
```



```
In [10]: #Plotting sample of anomalous data
plt.grid()
plt.plot(np.arange(1000), anomalous_train_data[0])
plt.title("An abnormal vibration signal (Air leakage)")
plt.show()
```



```
In [12]: #Creating the artificial neural network using Autoencoder
EMBEDDING_SIZE = 64 #Define how many neurons in the inner layer <-----
class AnomalyDetector(Model):
    def __init__(self):
        super(AnomalyDetector, self).__init__()
        self.encoder = tf.keras.Sequential([
            layers.Dense(32, activation="relu"),
            layers.Dense(16, activation="relu"),
```

```

        layers.Dense(EMBEDDING_SIZE, activation="relu"))] # Smallest Layer Defined Here

self.decoder = tf.keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(1000, activation="sigmoid")])

def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded

autoencoder = AnomalyDetector()
print("Chosen Embedding Size: ", EMBEDDING_SIZE)

autoencoder.compile(optimizer='adam', loss='mae')
#Training the model.
history = autoencoder.fit(normal_train_data, normal_train_data,
    epochs=200,
    batch_size=200,
    validation_data=(test_data, test_data),
    shuffle=True)

```

Chosen Embedding Size: 64

Epoch 1/200				
3/3		10s	811ms/step	- loss: 0.2426 - val_loss: 0.2424
Epoch 2/200				
3/3		1s	259ms/step	- loss: 0.2394 - val_loss: 0.2365
Epoch 3/200				
3/3		1s	207ms/step	- loss: 0.2315 - val_loss: 0.2252
Epoch 4/200				
3/3		1s	218ms/step	- loss: 0.2168 - val_loss: 0.2070
Epoch 5/200				
3/3		0s	144ms/step	- loss: 0.1944 - val_loss: 0.1844
Epoch 6/200				
3/3		0s	36ms/step	- loss: 0.1689 - val_loss: 0.1653
Epoch 7/200				
3/3		0s	133ms/step	- loss: 0.1498 - val_loss: 0.1542
Epoch 8/200				
3/3		0s	122ms/step	- loss: 0.1386 - val_loss: 0.1446
Epoch 9/200				
3/3		0s	127ms/step	- loss: 0.1270 - val_loss: 0.1328
Epoch 10/200				
3/3		1s	78ms/step	- loss: 0.1141 - val_loss: 0.1247
Epoch 11/200				
3/3		0s	86ms/step	- loss: 0.1068 - val_loss: 0.1224
Epoch 12/200				
3/3		0s	147ms/step	- loss: 0.1045 - val_loss: 0.1206
Epoch 13/200				
3/3		0s	114ms/step	- loss: 0.1023 - val_loss: 0.1190
Epoch 14/200				
3/3		0s	126ms/step	- loss: 0.1009 - val_loss: 0.1190
Epoch 15/200				
3/3		0s	132ms/step	- loss: 0.1010 - val_loss: 0.1187
Epoch 16/200				
3/3		0s	60ms/step	- loss: 0.1005 - val_loss: 0.1181
Epoch 17/200				
3/3		0s	86ms/step	- loss: 0.0999 - val_loss: 0.1180
Epoch 18/200				
3/3		0s	118ms/step	- loss: 0.0997 - val_loss: 0.1177
Epoch 19/200				
3/3		0s	79ms/step	- loss: 0.0994 - val_loss: 0.1175
Epoch 20/200				
3/3		0s	101ms/step	- loss: 0.0992 - val_loss: 0.1175
Epoch 21/200				
3/3		0s	106ms/step	- loss: 0.0992 - val_loss: 0.1174
Epoch 22/200				
3/3		0s	76ms/step	- loss: 0.0990 - val_loss: 0.1173
Epoch 23/200				
3/3		0s	106ms/step	- loss: 0.0990 - val_loss: 0.1173
Epoch 24/200				
3/3		0s	103ms/step	- loss: 0.0989 - val_loss: 0.1172
Epoch 25/200				
3/3		0s	82ms/step	- loss: 0.0989 - val_loss: 0.1172
Epoch 26/200				
3/3		0s	77ms/step	- loss: 0.0989 - val_loss: 0.1172
Epoch 27/200				
3/3		0s	61ms/step	- loss: 0.0989 - val_loss: 0.1172
Epoch 28/200				
3/3		0s	80ms/step	- loss: 0.0988 - val_loss: 0.1172
Epoch 29/200				
3/3		0s	144ms/step	- loss: 0.0988 - val_loss: 0.1172
Epoch 30/200				
3/3		0s	75ms/step	- loss: 0.0988 - val_loss: 0.1172
Epoch 31/200				
3/3		0s	120ms/step	- loss: 0.0988 - val_loss: 0.1171
Epoch 32/200				
3/3		0s	144ms/step	- loss: 0.0988 - val_loss: 0.1171
Epoch 33/200				
3/3		1s	210ms/step	- loss: 0.0988 - val_loss: 0.1171
Epoch 34/200				
3/3		0s	91ms/step	- loss: 0.0988 - val_loss: 0.1171
Epoch 35/200				

3/3  0s 90ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 36/200
3/3  0s 83ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 37/200
3/3  1s 217ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 38/200
3/3  0s 194ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 39/200
3/3  1s 191ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 40/200
3/3  0s 136ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 41/200
3/3  0s 132ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 42/200
3/3  1s 113ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 43/200
3/3  0s 120ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 44/200
3/3  0s 104ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 45/200
3/3  0s 123ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 46/200
3/3  0s 136ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 47/200
3/3  0s 88ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 48/200
3/3  0s 111ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 49/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 50/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 51/200
3/3  0s 35ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 52/200
3/3  0s 40ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 53/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 54/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 55/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 56/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 57/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 58/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 59/200
3/3  0s 35ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 60/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 61/200
3/3  0s 64ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 62/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 63/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 64/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 65/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 66/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 67/200
3/3  0s 38ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 68/200
3/3  0s 35ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 69/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 70/200

3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 71/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 72/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 73/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 74/200
3/3  0s 35ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 75/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 76/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 77/200
3/3  0s 35ms/step - loss: 0.0988 - val_loss: 0.1172
Epoch 78/200
3/3  0s 56ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 79/200
3/3  0s 38ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 80/200
3/3  0s 43ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 81/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 82/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 83/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 84/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 85/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 86/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 87/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 88/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 89/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 90/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 91/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 92/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 93/200
3/3  0s 36ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 94/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 95/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 96/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 97/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 98/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 99/200
3/3  0s 59ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 100/200
3/3  0s 37ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 101/200
3/3  0s 39ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 102/200
3/3  0s 38ms/step - loss: 0.0988 - val_loss: 0.1171
Epoch 103/200
3/3  0s 36ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 104/200
3/3  0s 36ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 105/200

3/3  0s 36ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 106/200
3/3  0s 36ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 107/200
3/3  0s 36ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 108/200
3/3  0s 43ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 109/200
3/3  0s 37ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 110/200
3/3  0s 38ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 111/200
3/3  0s 36ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 112/200
3/3  0s 36ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 113/200
3/3  0s 37ms/step - loss: 0.0987 - val_loss: 0.1171
Epoch 114/200
3/3  0s 37ms/step - loss: 0.0987 - val_loss: 0.1170
Epoch 115/200
3/3  0s 37ms/step - loss: 0.0987 - val_loss: 0.1170
Epoch 116/200
3/3  0s 36ms/step - loss: 0.0987 - val_loss: 0.1170
Epoch 117/200
3/3  0s 37ms/step - loss: 0.0987 - val_loss: 0.1170
Epoch 118/200
3/3  0s 38ms/step - loss: 0.0987 - val_loss: 0.1170
Epoch 119/200
3/3  0s 38ms/step - loss: 0.0987 - val_loss: 0.1170
Epoch 120/200
3/3  0s 37ms/step - loss: 0.0987 - val_loss: 0.1170
Epoch 121/200
3/3  0s 36ms/step - loss: 0.0986 - val_loss: 0.1170
Epoch 122/200
3/3  0s 37ms/step - loss: 0.0986 - val_loss: 0.1170
Epoch 123/200
3/3  0s 37ms/step - loss: 0.0986 - val_loss: 0.1169
Epoch 124/200
3/3  0s 37ms/step - loss: 0.0986 - val_loss: 0.1169
Epoch 125/200
3/3  0s 36ms/step - loss: 0.0986 - val_loss: 0.1169
Epoch 126/200
3/3  0s 70ms/step - loss: 0.0986 - val_loss: 0.1169
Epoch 127/200
3/3  0s 69ms/step - loss: 0.0985 - val_loss: 0.1168
Epoch 128/200
3/3  0s 53ms/step - loss: 0.0985 - val_loss: 0.1168
Epoch 129/200
3/3  0s 55ms/step - loss: 0.0985 - val_loss: 0.1168
Epoch 130/200
3/3  0s 54ms/step - loss: 0.0985 - val_loss: 0.1167
Epoch 131/200
3/3  0s 51ms/step - loss: 0.0984 - val_loss: 0.1167
Epoch 132/200
3/3  0s 66ms/step - loss: 0.0983 - val_loss: 0.1166
Epoch 133/200
3/3  0s 68ms/step - loss: 0.0983 - val_loss: 0.1165
Epoch 134/200
3/3  0s 51ms/step - loss: 0.0982 - val_loss: 0.1165
Epoch 135/200
3/3  0s 55ms/step - loss: 0.0981 - val_loss: 0.1164
Epoch 136/200
3/3  0s 66ms/step - loss: 0.0981 - val_loss: 0.1163
Epoch 137/200
3/3  0s 52ms/step - loss: 0.0980 - val_loss: 0.1162
Epoch 138/200
3/3  0s 71ms/step - loss: 0.0978 - val_loss: 0.1160
Epoch 139/200
3/3  0s 65ms/step - loss: 0.0976 - val_loss: 0.1158
Epoch 140/200

3/3  0s 74ms/step - loss: 0.0974 - val_loss: 0.1155
Epoch 141/200
3/3  0s 74ms/step - loss: 0.0972 - val_loss: 0.1152
Epoch 142/200
3/3  0s 47ms/step - loss: 0.0969 - val_loss: 0.1148
Epoch 143/200
3/3  0s 37ms/step - loss: 0.0965 - val_loss: 0.1144
Epoch 144/200
3/3  0s 36ms/step - loss: 0.0961 - val_loss: 0.1139
Epoch 145/200
3/3  0s 38ms/step - loss: 0.0956 - val_loss: 0.1133
Epoch 146/200
3/3  0s 40ms/step - loss: 0.0950 - val_loss: 0.1127
Epoch 147/200
3/3  0s 36ms/step - loss: 0.0945 - val_loss: 0.1123
Epoch 148/200
3/3  0s 36ms/step - loss: 0.0940 - val_loss: 0.1114
Epoch 149/200
3/3  0s 36ms/step - loss: 0.0934 - val_loss: 0.1109
Epoch 150/200
3/3  0s 36ms/step - loss: 0.0928 - val_loss: 0.1103
Epoch 151/200
3/3  0s 36ms/step - loss: 0.0923 - val_loss: 0.1098
Epoch 152/200
3/3  0s 36ms/step - loss: 0.0919 - val_loss: 0.1093
Epoch 153/200
3/3  0s 37ms/step - loss: 0.0916 - val_loss: 0.1090
Epoch 154/200
3/3  0s 38ms/step - loss: 0.0912 - val_loss: 0.1088
Epoch 155/200
3/3  0s 37ms/step - loss: 0.0910 - val_loss: 0.1085
Epoch 156/200
3/3  0s 39ms/step - loss: 0.0907 - val_loss: 0.1083
Epoch 157/200
3/3  0s 35ms/step - loss: 0.0905 - val_loss: 0.1082
Epoch 158/200
3/3  0s 36ms/step - loss: 0.0904 - val_loss: 0.1082
Epoch 159/200
3/3  0s 36ms/step - loss: 0.0904 - val_loss: 0.1080
Epoch 160/200
3/3  0s 39ms/step - loss: 0.0903 - val_loss: 0.1080
Epoch 161/200
3/3  0s 37ms/step - loss: 0.0902 - val_loss: 0.1079
Epoch 162/200
3/3  0s 36ms/step - loss: 0.0902 - val_loss: 0.1080
Epoch 163/200
3/3  0s 37ms/step - loss: 0.0902 - val_loss: 0.1079
Epoch 164/200
3/3  0s 36ms/step - loss: 0.0901 - val_loss: 0.1078
Epoch 165/200
3/3  0s 42ms/step - loss: 0.0901 - val_loss: 0.1078
Epoch 166/200
3/3  0s 36ms/step - loss: 0.0901 - val_loss: 0.1078
Epoch 167/200
3/3  0s 36ms/step - loss: 0.0901 - val_loss: 0.1078
Epoch 168/200
3/3  0s 37ms/step - loss: 0.0900 - val_loss: 0.1078
Epoch 169/200
3/3  0s 36ms/step - loss: 0.0900 - val_loss: 0.1079
Epoch 170/200
3/3  0s 37ms/step - loss: 0.0901 - val_loss: 0.1079
Epoch 171/200
3/3  0s 37ms/step - loss: 0.0900 - val_loss: 0.1078
Epoch 172/200
3/3  0s 38ms/step - loss: 0.0900 - val_loss: 0.1077
Epoch 173/200
3/3  0s 37ms/step - loss: 0.0899 - val_loss: 0.1077
Epoch 174/200
3/3  0s 43ms/step - loss: 0.0899 - val_loss: 0.1077
Epoch 175/200

```

3/3 ————— 0s 36ms/step - loss: 0.0899 - val_loss: 0.1077
Epoch 176/200
3/3 ————— 0s 37ms/step - loss: 0.0899 - val_loss: 0.1077
Epoch 177/200
3/3 ————— 0s 37ms/step - loss: 0.0899 - val_loss: 0.1077
Epoch 178/200
3/3 ————— 0s 35ms/step - loss: 0.0898 - val_loss: 0.1077
Epoch 179/200
3/3 ————— 0s 37ms/step - loss: 0.0899 - val_loss: 0.1078
Epoch 180/200
3/3 ————— 0s 39ms/step - loss: 0.0899 - val_loss: 0.1077
Epoch 181/200
3/3 ————— 0s 37ms/step - loss: 0.0898 - val_loss: 0.1077
Epoch 182/200
3/3 ————— 0s 39ms/step - loss: 0.0898 - val_loss: 0.1076
Epoch 183/200
3/3 ————— 0s 37ms/step - loss: 0.0898 - val_loss: 0.1077
Epoch 184/200
3/3 ————— 0s 37ms/step - loss: 0.0898 - val_loss: 0.1076
Epoch 185/200
3/3 ————— 0s 37ms/step - loss: 0.0898 - val_loss: 0.1077
Epoch 186/200
3/3 ————— 0s 37ms/step - loss: 0.0899 - val_loss: 0.1076
Epoch 187/200
3/3 ————— 0s 37ms/step - loss: 0.0898 - val_loss: 0.1076
Epoch 188/200
3/3 ————— 0s 37ms/step - loss: 0.0897 - val_loss: 0.1076
Epoch 189/200
3/3 ————— 0s 38ms/step - loss: 0.0897 - val_loss: 0.1076
Epoch 190/200
3/3 ————— 0s 37ms/step - loss: 0.0897 - val_loss: 0.1075
Epoch 191/200
3/3 ————— 0s 38ms/step - loss: 0.0897 - val_loss: 0.1075
Epoch 192/200
3/3 ————— 0s 38ms/step - loss: 0.0897 - val_loss: 0.1075
Epoch 193/200
3/3 ————— 0s 44ms/step - loss: 0.0896 - val_loss: 0.1075
Epoch 194/200
3/3 ————— 0s 37ms/step - loss: 0.0896 - val_loss: 0.1075
Epoch 195/200
3/3 ————— 0s 37ms/step - loss: 0.0896 - val_loss: 0.1075
Epoch 196/200
3/3 ————— 0s 36ms/step - loss: 0.0895 - val_loss: 0.1074
Epoch 197/200
3/3 ————— 0s 39ms/step - loss: 0.0895 - val_loss: 0.1074
Epoch 198/200
3/3 ————— 0s 36ms/step - loss: 0.0894 - val_loss: 0.1072
Epoch 199/200
3/3 ————— 0s 38ms/step - loss: 0.0894 - val_loss: 0.1072
Epoch 200/200
3/3 ————— 0s 38ms/step - loss: 0.0893 - val_loss: 0.1071

```

```

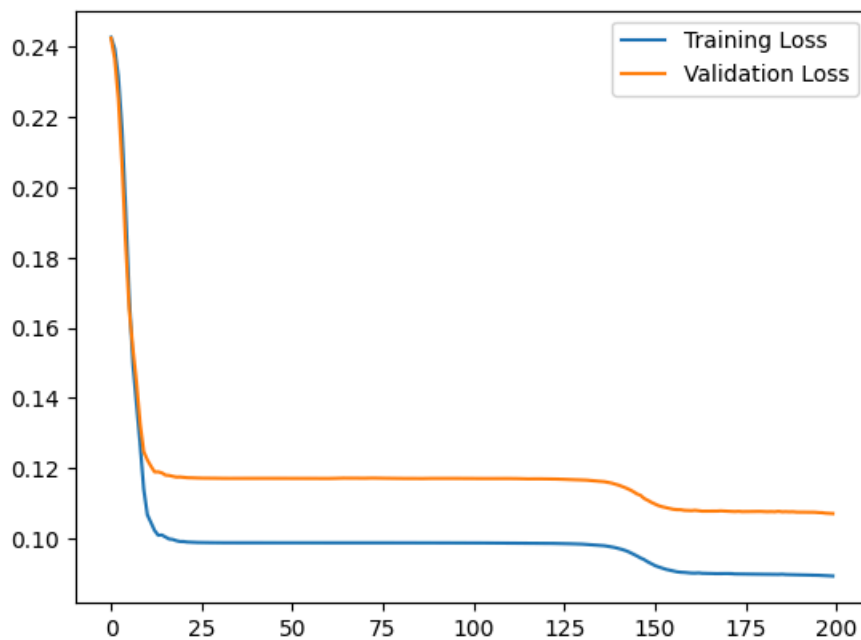
In [13]: #Plotting the evolution of training and validation Loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.legend()

```

```

Out[13]: <matplotlib.legend.Legend at 0x7a047f4bfff80>

```

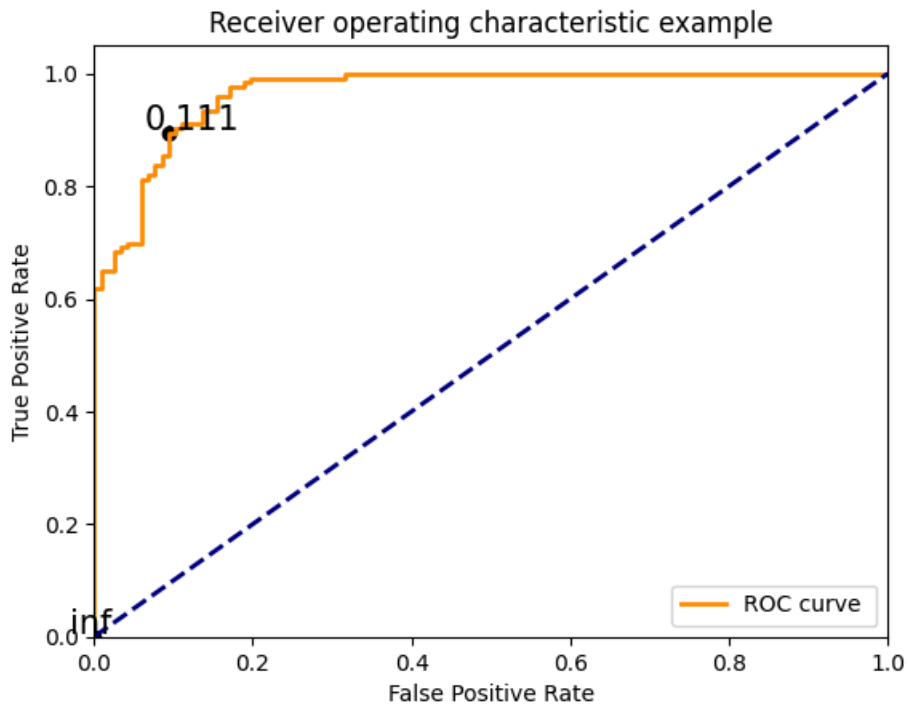


How are the loss functions looking? Is there a need to adjust the EMBEDDING SIZE or the epochs in order to minimize it more?

```
In [14]: #Plotting True positive and false positive rate assessment
reconstructions = autoencoder(test_data)
loss = tf.keras.losses.mae(reconstructions, test_data)
fpr = []
tpr = []
#the test Labels are flipped to match how the roc_curve function expects them.
flipped_labels = 1-test_labels
fpr, tpr, thresholds = roc_curve(flipped_labels, loss)
plt.figure()
lw = 2
plt.plot(fpr, tpr, color='darkorange',
         lw=lw, label='ROC curve ')
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc="lower right")

# plot some thresholds
thresholds_every=20
thresholdsLength = len(thresholds)
colorMap=plt.get_cmap('jet', thresholdsLength)
for i in range(0, thresholdsLength, thresholds_every):
    threshold_value_with_max_four_decimals = str(thresholds[i])[:5]
    plt.scatter(fpr[i], tpr[i], c='black')
    plt.text(fpr[i] - 0.03, tpr[i] + 0.005, threshold_value_with_max_four_decimals, fontdict={'size': 15});

plt.show()
```



```
In [15]: roc_auc = auc(fpr, tpr)
         print(roc_auc)
```

```
0.9671322354249183
```

```
In [17]: threshold = 0.111 #Assign a value labeled in black in the ROC graph <-----
def predict(model, data, threshold):
    reconstructions = model(data)
    loss = tf.keras.losses.mae(reconstructions, data)
    return tf.math.less(loss, threshold), loss

def print_stats(predictions, labels):
    print("Accuracy = {}".format(accuracy_score(labels, predictions)))
    print("Precision = {}".format(precision_score(labels, predictions)))
    print("Recall = {}".format(recall_score(labels, predictions)))

preds, scores = predict(autoencoder, test_data, threshold)
print_stats(preds, test_labels)
```

```
Accuracy = 0.9
Precision = 0.8907563025210085
Recall = 0.905982905982906
```

Task 3.1

How can you compare the models using data from the X-axis and Y-axis data? which one does a better job classifying?
Explain your reasoning.

The Y-axis data does a much better job of classifying since it actually shows a knee in the ROC plot and varies with the embedding size.

Working in the Z-axis

Recycle the code from the previous two dimensions, to build a model using the data from the Z-axis.

In [19]: *#Your code here*

```
AXIS = 'Zacc array [m/s2]' #Pick and write the axis you want to work <-----

#Exploding the values contained in selected column and converting the string values into float values
df_new = pd.concat([df['Condition'], df[AXIS].str.split(' ', expand=True).astype(float)], axis=1)
ds = df_new.copy()
#Converting the Classifier into binary values
ds.loc[df['Condition'] == 'Vacuuming', 'Status'] = 1
ds.loc[df['Condition'] == 'Air_leakage', 'Status'] = 0
ds.drop('Condition', axis=1, inplace=True)

#Data transformation

raw_data = ds.values
# The Last element contains the labels
labels = raw_data[:, -1]

# The other data points are the vacuum accelerometer data
data = raw_data[:, 0:-1]

train_data, test_data, train_labels, test_labels = train_test_split(
    data, labels, test_size=0.2, random_state=21
)
#Normalizing the values of the dataset
min_val = tf.reduce_min(train_data)
max_val = tf.reduce_max(train_data)

train_data = (train_data - min_val) / (max_val - min_val)
test_data = (test_data - min_val) / (max_val - min_val)

train_data = tf.cast(train_data, tf.float32)
test_data = tf.cast(test_data, tf.float32)
#Splitting the dataset based on classification: train_Labels: Vacuuming, ~train_Labels: Air Leakage
train_labels = train_labels.astype(bool)
test_labels = test_labels.astype(bool)

normal_train_data = train_data[train_labels]
normal_test_data = test_data[test_labels]

anomalous_train_data = train_data[~train_labels]
anomalous_test_data = test_data[~test_labels]

portion_of_anomaly_in_training = 0.1 #10% of training data will be anomalies
end_size = int(len(normal_train_data)/(10-portion_of_anomaly_in_training*10))
combined_train_data = np.append(normal_train_data, anomalous_test_data[:end_size], axis=0)
combined_train_data.shape

# Anomaly Detector

autoencoderZ = AnomalyDetector()
print("Chosen Embedding Size: ", EMBEDDING_SIZE)

autoencoderZ.compile(optimizer='adam', loss='mae')
#Training the model.
history = autoencoderZ.fit(normal_train_data, normal_train_data,
    epochs=200,
    batch_size=200,
    validation_data=(test_data, test_data),
    shuffle=True)

#Plotting the evolution of training and validation Loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.legend()

#Plotting True positive and false positive rate assessment
reconstructions = autoencoderZ(test_data)
loss = tf.keras.losses.mae(reconstructions, test_data)
```

```

fpr = []
tpr = []
#the test labels are flipped to match how the roc_curve function expects them.
flipped_labels = 1-test_labels
fpr, tpr, thresholds = roc_curve(flipped_labels, loss)
plt.figure()
lw = 2
plt.plot(fpr, tpr, color='darkorange',
         lw=lw, label='ROC curve ')
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc="lower right")

# plot some thresholds
thresholds_every=20
thresholdsLength = len(thresholds)
colorMap=plt.get_cmap('jet', thresholdsLength)
for i in range(0, thresholdsLength, thresholds_every):
    threshold_value_with_max_four_decimals = str(thresholds[i])[:5]
    plt.scatter(fpr[i], tpr[i], c='black')
    plt.text(fpr[i] - 0.03, tpr[i] + 0.005, threshold_value_with_max_four_decimals, fontdict={'size': 15});

plt.show()

#

```

```

Chosen Embedding Size: 64
Epoch 1/200
3/3  0s 96ms/step - loss: 0.1596 - val_loss: 0.0976
Epoch 2/200
3/3  0s 109ms/step - loss: 0.0816 - val_loss: 0.0681
Epoch 3/200
3/3  0s 94ms/step - loss: 0.0629 - val_loss: 0.0661
Epoch 4/200
3/3  0s 88ms/step - loss: 0.0615 - val_loss: 0.0655
Epoch 5/200
3/3  0s 71ms/step - loss: 0.0612 - val_loss: 0.0652
Epoch 6/200
3/3  0s 112ms/step - loss: 0.0610 - val_loss: 0.0650
Epoch 7/200
3/3  1s 309ms/step - loss: 0.0609 - val_loss: 0.0648
Epoch 8/200
3/3  1s 231ms/step - loss: 0.0606 - val_loss: 0.0645
Epoch 9/200
3/3  1s 195ms/step - loss: 0.0603 - val_loss: 0.0642
Epoch 10/200
3/3  1s 217ms/step - loss: 0.0599 - val_loss: 0.0642
Epoch 11/200
3/3  0s 166ms/step - loss: 0.0598 - val_loss: 0.0642
Epoch 12/200
3/3  0s 136ms/step - loss: 0.0596 - val_loss: 0.0640
Epoch 13/200
3/3  0s 150ms/step - loss: 0.0595 - val_loss: 0.0639
Epoch 14/200
3/3  0s 128ms/step - loss: 0.0594 - val_loss: 0.0639
Epoch 15/200
3/3  0s 121ms/step - loss: 0.0594 - val_loss: 0.0639
Epoch 16/200
3/3  0s 154ms/step - loss: 0.0593 - val_loss: 0.0639
Epoch 17/200
3/3  0s 155ms/step - loss: 0.0593 - val_loss: 0.0639
Epoch 18/200
3/3  0s 106ms/step - loss: 0.0593 - val_loss: 0.0639
Epoch 19/200
3/3  0s 106ms/step - loss: 0.0592 - val_loss: 0.0638
Epoch 20/200
3/3  0s 110ms/step - loss: 0.0592 - val_loss: 0.0638
Epoch 21/200
3/3  0s 97ms/step - loss: 0.0592 - val_loss: 0.0638
Epoch 22/200
3/3  0s 113ms/step - loss: 0.0592 - val_loss: 0.0638
Epoch 23/200
3/3  0s 68ms/step - loss: 0.0592 - val_loss: 0.0638
Epoch 24/200
3/3  0s 85ms/step - loss: 0.0592 - val_loss: 0.0638
Epoch 25/200
3/3  0s 95ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 26/200
3/3  0s 81ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 27/200
3/3  0s 154ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 28/200
3/3  0s 89ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 29/200
3/3  0s 74ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 30/200
3/3  0s 64ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 31/200
3/3  0s 50ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 32/200
3/3  0s 35ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 33/200
3/3  0s 35ms/step - loss: 0.0591 - val_loss: 0.0638
Epoch 34/200
3/3  0s 40ms/step - loss: 0.0590 - val_loss: 0.0638
Epoch 35/200

```

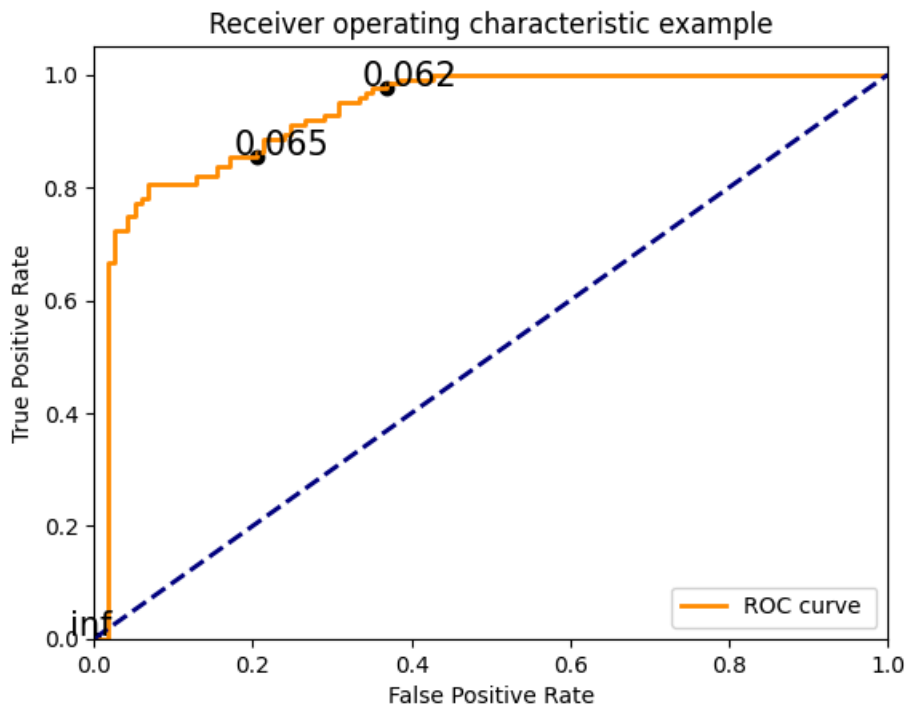
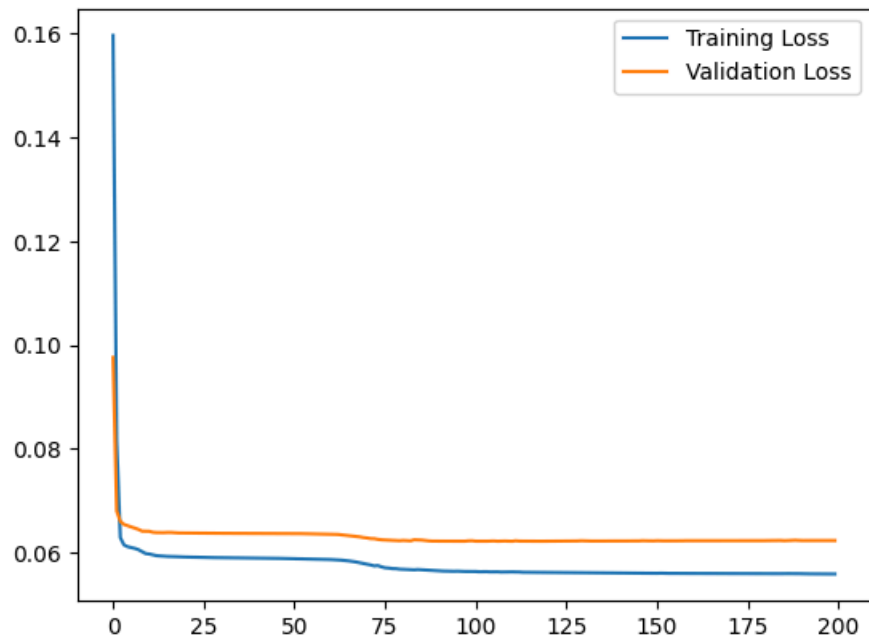
3/3  0s 36ms/step - loss: 0.0590 - val_loss: 0.0638
Epoch 36/200
3/3  0s 35ms/step - loss: 0.0590 - val_loss: 0.0638
Epoch 37/200
3/3  0s 35ms/step - loss: 0.0590 - val_loss: 0.0638
Epoch 38/200
3/3  0s 36ms/step - loss: 0.0590 - val_loss: 0.0638
Epoch 39/200
3/3  0s 36ms/step - loss: 0.0590 - val_loss: 0.0637
Epoch 40/200
3/3  0s 35ms/step - loss: 0.0590 - val_loss: 0.0637
Epoch 41/200
3/3  0s 35ms/step - loss: 0.0590 - val_loss: 0.0637
Epoch 42/200
3/3  0s 35ms/step - loss: 0.0590 - val_loss: 0.0637
Epoch 43/200
3/3  0s 42ms/step - loss: 0.0590 - val_loss: 0.0637
Epoch 44/200
3/3  0s 36ms/step - loss: 0.0590 - val_loss: 0.0637
Epoch 45/200
3/3  0s 35ms/step - loss: 0.0590 - val_loss: 0.0637
Epoch 46/200
3/3  0s 36ms/step - loss: 0.0590 - val_loss: 0.0637
Epoch 47/200
3/3  0s 37ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 48/200
3/3  0s 36ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 49/200
3/3  0s 36ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 50/200
3/3  0s 36ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 51/200
3/3  0s 37ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 52/200
3/3  0s 37ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 53/200
3/3  0s 36ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 54/200
3/3  0s 36ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 55/200
3/3  0s 36ms/step - loss: 0.0589 - val_loss: 0.0637
Epoch 56/200
3/3  0s 36ms/step - loss: 0.0588 - val_loss: 0.0637
Epoch 57/200
3/3  0s 35ms/step - loss: 0.0588 - val_loss: 0.0637
Epoch 58/200
3/3  0s 35ms/step - loss: 0.0588 - val_loss: 0.0637
Epoch 59/200
3/3  0s 36ms/step - loss: 0.0588 - val_loss: 0.0636
Epoch 60/200
3/3  0s 36ms/step - loss: 0.0588 - val_loss: 0.0636
Epoch 61/200
3/3  0s 36ms/step - loss: 0.0587 - val_loss: 0.0636
Epoch 62/200
3/3  0s 36ms/step - loss: 0.0587 - val_loss: 0.0636
Epoch 63/200
3/3  0s 36ms/step - loss: 0.0586 - val_loss: 0.0636
Epoch 64/200
3/3  0s 72ms/step - loss: 0.0586 - val_loss: 0.0635
Epoch 65/200
3/3  0s 52ms/step - loss: 0.0585 - val_loss: 0.0634
Epoch 66/200
3/3  0s 53ms/step - loss: 0.0585 - val_loss: 0.0634
Epoch 67/200
3/3  0s 54ms/step - loss: 0.0584 - val_loss: 0.0633
Epoch 68/200
3/3  0s 67ms/step - loss: 0.0583 - val_loss: 0.0632
Epoch 69/200
3/3  0s 53ms/step - loss: 0.0581 - val_loss: 0.0631
Epoch 70/200

3/3  0s 69ms/step - loss: 0.0580 - val_loss: 0.0630
Epoch 71/200
3/3  0s 54ms/step - loss: 0.0578 - val_loss: 0.0629
Epoch 72/200
3/3  0s 71ms/step - loss: 0.0577 - val_loss: 0.0628
Epoch 73/200
3/3  0s 66ms/step - loss: 0.0575 - val_loss: 0.0627
Epoch 74/200
3/3  0s 71ms/step - loss: 0.0576 - val_loss: 0.0626
Epoch 75/200
3/3  0s 57ms/step - loss: 0.0573 - val_loss: 0.0625
Epoch 76/200
3/3  0s 73ms/step - loss: 0.0571 - val_loss: 0.0625
Epoch 77/200
3/3  0s 37ms/step - loss: 0.0571 - val_loss: 0.0624
Epoch 78/200
3/3  0s 35ms/step - loss: 0.0570 - val_loss: 0.0624
Epoch 79/200
3/3  0s 36ms/step - loss: 0.0570 - val_loss: 0.0624
Epoch 80/200
3/3  0s 36ms/step - loss: 0.0569 - val_loss: 0.0623
Epoch 81/200
3/3  0s 42ms/step - loss: 0.0568 - val_loss: 0.0624
Epoch 82/200
3/3  0s 36ms/step - loss: 0.0568 - val_loss: 0.0624
Epoch 83/200
3/3  0s 37ms/step - loss: 0.0568 - val_loss: 0.0623
Epoch 84/200
3/3  0s 35ms/step - loss: 0.0567 - val_loss: 0.0625
Epoch 85/200
3/3  0s 36ms/step - loss: 0.0568 - val_loss: 0.0625
Epoch 86/200
3/3  0s 35ms/step - loss: 0.0568 - val_loss: 0.0625
Epoch 87/200
3/3  0s 36ms/step - loss: 0.0567 - val_loss: 0.0624
Epoch 88/200
3/3  0s 36ms/step - loss: 0.0567 - val_loss: 0.0623
Epoch 89/200
3/3  0s 35ms/step - loss: 0.0566 - val_loss: 0.0623
Epoch 90/200
3/3  0s 35ms/step - loss: 0.0566 - val_loss: 0.0623
Epoch 91/200
3/3  0s 42ms/step - loss: 0.0565 - val_loss: 0.0623
Epoch 92/200
3/3  0s 38ms/step - loss: 0.0565 - val_loss: 0.0623
Epoch 93/200
3/3  0s 34ms/step - loss: 0.0565 - val_loss: 0.0623
Epoch 94/200
3/3  0s 35ms/step - loss: 0.0565 - val_loss: 0.0623
Epoch 95/200
3/3  0s 35ms/step - loss: 0.0565 - val_loss: 0.0623
Epoch 96/200
3/3  0s 36ms/step - loss: 0.0565 - val_loss: 0.0623
Epoch 97/200
3/3  0s 36ms/step - loss: 0.0564 - val_loss: 0.0622
Epoch 98/200
3/3  0s 36ms/step - loss: 0.0564 - val_loss: 0.0623
Epoch 99/200
3/3  0s 36ms/step - loss: 0.0564 - val_loss: 0.0623
Epoch 100/200
3/3  0s 35ms/step - loss: 0.0565 - val_loss: 0.0623
Epoch 101/200
3/3  0s 43ms/step - loss: 0.0564 - val_loss: 0.0623
Epoch 102/200
3/3  0s 37ms/step - loss: 0.0564 - val_loss: 0.0622
Epoch 103/200
3/3  0s 36ms/step - loss: 0.0564 - val_loss: 0.0623
Epoch 104/200
3/3  0s 35ms/step - loss: 0.0564 - val_loss: 0.0623
Epoch 105/200

3/3  0s 36ms/step - loss: 0.0564 - val_loss: 0.0623
Epoch 106/200
3/3  0s 36ms/step - loss: 0.0564 - val_loss: 0.0623
Epoch 107/200
3/3  0s 36ms/step - loss: 0.0564 - val_loss: 0.0622
Epoch 108/200
3/3  0s 37ms/step - loss: 0.0563 - val_loss: 0.0623
Epoch 109/200
3/3  0s 37ms/step - loss: 0.0563 - val_loss: 0.0623
Epoch 110/200
3/3  0s 38ms/step - loss: 0.0563 - val_loss: 0.0623
Epoch 111/200
3/3  0s 40ms/step - loss: 0.0564 - val_loss: 0.0622
Epoch 112/200
3/3  0s 36ms/step - loss: 0.0563 - val_loss: 0.0623
Epoch 113/200
3/3  0s 36ms/step - loss: 0.0563 - val_loss: 0.0623
Epoch 114/200
3/3  0s 36ms/step - loss: 0.0563 - val_loss: 0.0623
Epoch 115/200
3/3  0s 36ms/step - loss: 0.0563 - val_loss: 0.0623
Epoch 116/200
3/3  0s 37ms/step - loss: 0.0563 - val_loss: 0.0623
Epoch 117/200
3/3  0s 37ms/step - loss: 0.0562 - val_loss: 0.0622
Epoch 118/200
3/3  0s 35ms/step - loss: 0.0562 - val_loss: 0.0622
Epoch 119/200
3/3  0s 36ms/step - loss: 0.0562 - val_loss: 0.0622
Epoch 120/200
3/3  0s 43ms/step - loss: 0.0562 - val_loss: 0.0622
Epoch 121/200
3/3  0s 36ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 122/200
3/3  0s 35ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 123/200
3/3  0s 36ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 124/200
3/3  0s 37ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 125/200
3/3  0s 38ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 126/200
3/3  0s 37ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 127/200
3/3  0s 36ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 128/200
3/3  0s 35ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 129/200
3/3  0s 35ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 130/200
3/3  0s 38ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 131/200
3/3  0s 37ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 132/200
3/3  0s 37ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 133/200
3/3  0s 36ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 134/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 135/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 136/200
3/3  0s 34ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 137/200
3/3  0s 35ms/step - loss: 0.0562 - val_loss: 0.0623
Epoch 138/200
3/3  0s 35ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 139/200
3/3  0s 42ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 140/200

3/3  0s 38ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 141/200
3/3  0s 39ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 142/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 143/200
3/3  0s 37ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 144/200
3/3  0s 35ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 145/200
3/3  0s 37ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 146/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 147/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 148/200
3/3  0s 37ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 149/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 150/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 151/200
3/3  0s 37ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 152/200
3/3  0s 38ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 153/200
3/3  0s 38ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 154/200
3/3  0s 39ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 155/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 156/200
3/3  0s 37ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 157/200
3/3  0s 37ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 158/200
3/3  0s 45ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 159/200
3/3  0s 39ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 160/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 161/200
3/3  0s 37ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 162/200
3/3  0s 36ms/step - loss: 0.0561 - val_loss: 0.0623
Epoch 163/200
3/3  0s 38ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 164/200
3/3  0s 36ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 165/200
3/3  0s 36ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 166/200
3/3  0s 39ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 167/200
3/3  0s 56ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 168/200
3/3  0s 38ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 169/200
3/3  0s 61ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 170/200
3/3  0s 55ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 171/200
3/3  0s 51ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 172/200
3/3  0s 52ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 173/200
3/3  0s 65ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 174/200
3/3  0s 53ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 175/200

3/3  0s 71ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 176/200
3/3  0s 52ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 177/200
3/3  0s 59ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 178/200
3/3  0s 76ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 179/200
3/3  0s 69ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 180/200
3/3  0s 70ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 181/200
3/3  0s 60ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 182/200
3/3  0s 39ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 183/200
3/3  0s 37ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 184/200
3/3  0s 43ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 185/200
3/3  0s 36ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 186/200
3/3  0s 38ms/step - loss: 0.0560 - val_loss: 0.0623
Epoch 187/200
3/3  0s 36ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 188/200
3/3  0s 39ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 189/200
3/3  0s 37ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 190/200
3/3  0s 39ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 191/200
3/3  0s 39ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 192/200
3/3  0s 38ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 193/200
3/3  0s 58ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 194/200
3/3  0s 36ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 195/200
3/3  0s 36ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 196/200
3/3  0s 37ms/step - loss: 0.0560 - val_loss: 0.0624
Epoch 197/200
3/3  0s 39ms/step - loss: 0.0559 - val_loss: 0.0624
Epoch 198/200
3/3  0s 38ms/step - loss: 0.0559 - val_loss: 0.0624
Epoch 199/200
3/3  0s 37ms/step - loss: 0.0559 - val_loss: 0.0624
Epoch 200/200
3/3  0s 36ms/step - loss: 0.0559 - val_loss: 0.0624



```
In [21]: threshold = 0.062 #Assign a value labeled in black in the ROC graph <-----
def predict(model, data, threshold):
    reconstructions = model(data)
    loss = tf.keras.losses.mae(reconstructions, data)
    return tf.math.less(loss, threshold), loss

def print_stats(predictions, labels):
    print("Accuracy = {}".format(accuracy_score(labels, predictions)))
    print("Precision = {}".format(precision_score(labels, predictions)))
    print("Recall = {}".format(recall_score(labels, predictions)))

preds, scores = predict(autoencoder, test_data, threshold)
print_stats(preds, test_labels)
```

Accuracy = 0.8625
 Precision = 0.875
 Recall = 0.8376068376068376

Task 3.2

Which model (X, Y, or Z) would you choose to classify normal and abnormal readings for the vacuum problem? Explain your reasoning.

I would choose a Y-axis model to classify normal and abnormal readings for the vacuum problem, since I was able to generate a proper ROC plot with this dataset as opposed to the X-axis model, and it proved to be more accurate and have more precision and recall when compared to the Z-axis model.

Task 3.3

What other data transformations/extractions would you consider to build a model to classify normal and abnormal data on the vacuum problem?

To build a model to classify normal and abnormal data I would also consider taking sound frequency data from a microphone placed near the vacuum. I would also want to compare the pressure or airflow at either end of the pump in both the air leak or normal operation.

Get back to [Lab Index Page](#)